

Building Production-Ready AI Agents

Without frameworks. Without magic. Just clean, debuggable Python.

Python	No Frameworks	Production Ready
--------	---------------	------------------

Why Skip the Frameworks?

Frameworks like AutoGen, CrewAI, and LangChain promise to make building AI agents easy. And for quick demos, they do. But when you need to ship something real — something that runs reliably at 3 AM without you babysitting it — they create more problems than they solve.

■ Blackbox Behavior You can't see what's happening inside. Debugging becomes guesswork.	■ Cognitive Overhead Frameworks force 'agents & crews' thinking instead of focusing on your actual problem.
■ Broken Logging Normal print() and logging calls don't work reliably inside framework loops.	■ No Guardrails AutoGen and similar tools provide no safety checks to prevent dangerous actions.
■ Memory Leaks All major frameworks suffer from uncontrolled resource consumption over time.	■ Limited Flexibility Framework abstractions get in the way the moment your use case is slightly unusual.

The Solution: Plain Python

Instead of wrapping everything in framework abstractions, write your agent as a simple Python pipeline. Every step is a function. Every function has clear inputs and outputs. You can read it, debug it, and test it like any other code.

1 • Direct Orchestration

Every step in your agent's logic should be explicit and easy to follow. No hidden retries, no automatic tool calls, no surprise behavior.

Explicit steps	Each action the agent takes is a named function call you wrote
Visible state	All data lives in a plain dict or dataclass — inspect it anytime
Easy to debug	Add a print() anywhere and it just works — no framework magic

```
# Clear, readable orchestration
def run_agent(user_input: str) -> str:
    state = {'input': user_input, 'history': []}
    plan = plan_task(state)
    result = execute_plan(state, plan)
    return format_output(state, result)
```

2 · Explicit State Management

Keep your agent's memory in plain Python objects. A dictionary is perfectly fine for most agents.

Dictionary	Use a plain dict or @dataclass — no ORM, no vector store needed to start
Explicit I/O	Every function takes state in and returns updated state out
Visible	You can log, print, or serialize the entire state at any point

```
from dataclasses import dataclass, field
from typing import List

@dataclass
class AgentState:
    task: str
    steps_taken: List[str] = field(default_factory=list)
    context: dict = field(default_factory=dict)
    is_complete: bool = False
```

3 · Direct API Calls

Call the LLM directly using the official SDK or a simple HTTP client. Handle retries, timeouts, and rate limits yourself — it's only a few lines.

Retry logic	Use tenacity or a simple loop — you control exactly when to retry
Timeouts	Set explicit timeout= on every request so a slow API never hangs your app
Rate limits	Catch 429 errors and sleep — no hidden queue or surprise backpressure
Error handling	Explicit try/except means you decide what happens on every failure

```
import anthropic, time

def call_llm(prompt: str, retries: int = 3) -> str:
    client = anthropic.Anthropic()
    for attempt in range(retries):
        try:
            msg = client.messages.create(
                model='claude-sonnet-4-20250514',
                max_tokens=1024,
                messages=[{'role': 'user', 'content': prompt}]
            )
            return msg.content[0].text
        except anthropic.RateLimitError:
            time.sleep(2 ** attempt) # exponential back-off
    raise RuntimeError('LLM call failed after retries')
```

4 · Pipeline Orchestration

Build your agent as a sequence of small, single-purpose functions. Each stage reads from state, does one thing, and writes back to state.

```
import logging
log = logging.getLogger(__name__)

def pipeline(user_input: str) -> str:
    state = AgentState(task=user_input)

    # Stage 1 - understand the request
    state.context['intent'] = classify_intent(state)
    log.info('Intent: %s', state.context['intent'])

    # Stage 2 - gather information
    state.context['data'] = fetch_relevant_data(state)
    log.info('Fetched %d items', len(state.context['data']))

    # Stage 3 - generate the answer
    answer = generate_response(state)
    log.info('Done. Steps taken: %s', state.steps_taken)
    return answer
```

5 · Logging That Actually Works

Because you're writing plain Python, Python's built-in logging module works perfectly. Set it up once at startup and every stage logs automatically.

```
import logging, sys

logging.basicConfig(
    level = logging.INFO,
    format = '%(asctime)s | %(levelname)s | %(message)s',
    handlers= [logging.StreamHandler(sys.stdout)]
)

# Every stage can now log freely
log = logging.getLogger(__name__)
log.info('Agent started for task: %s', task_id)
```

6 · Adding Your Own Guardrails

Frameworks give you nothing here. With plain Python you can add safety checks exactly where you need them — before a tool runs, before an API call, or before a result is returned.

```
BLOCKED = ['rm -rf', 'DROP TABLE', 'os.system']

def safe_execute(action: str) -> str:
    if any(b in action for b in BLOCKED):
        log.warning('Blocked dangerous action: %s', action)
        return 'Action blocked by safety check.'
    return execute(action)
```

7 · Controlling Memory Usage

Frameworks accumulate state silently and never clean up. With plain Python you decide exactly what to keep and for how long.

Cap history	Keep only the last N messages in context — trim in your state update function
Summarise often	Call the LLM to compress old context into a one-paragraph summary
Delete when done	At the end of a run, del state or let it go out of scope naturally
Profile if needed	tracemalloc and memory_profiler work normally — no framework interference

Quick-Reference Checklist

■	State	All agent state lives in a plain dict or dataclass
■	LLM calls	Wrapped in a function with retry, timeout, and explicit error handling
■	Pipeline	Each stage is a separate function with a single responsibility
■	Logging	logging.basicConfig() called once; log.info() used throughout
■	Guardrails	Validation functions run before any tool call or external action
■	Memory	History trimmed to a fixed window; state cleaned up after each run

■ **Start here:** Write one plain Python function that calls the LLM and returns a string. Then wrap it in a state dict. Then add logging. Then add retries. Build up layer by layer — you'll have a production agent faster than any framework tutorial gets you.